

Interpretable Traffic Sign Recognition: A Dual-Stage Pipeline Integrating YOLOv8 and Prototypical Decision Trees

1st Dario Bandecchi

Department of Information Engineering (DII)
University of Pisa
Pisa, Italy
d.bandecchi@studenti.unipi.it

2nd Gabriele Caioli

Department of Information Engineering (DII)
University of Pisa
Pisa, Italy
g.caioli@studenti.unipi.it

Abstract—In this work, we present an interpretable pipeline for traffic sign detection and classification combining object detection with prototype-based explainable AI. A YOLOv8 detector is fine-tuned on a dataset of approximately 14,000 annotated images spanning 200 traffic sign classes to localize signs in real-world driving scenes. Model selection follows a three-phase strategy: an initial screening across five YOLOv8 variants (nano to extra-large) on a single fold identifies the best accuracy-speed trade-off, a hyperparameter grid search refines the learning rate and backbone freezing strategy on the selected variant, and a final stratified k -fold cross-validation provides a robust performance estimate before evaluation on a held-out test set. To address the lack of interpretability in standard detection classifiers, a ProtoTree model is trained on a curated subset of 25 sign classes, chosen for their relevance to road safety, acting as an explanatory layer alongside the detector’s own classification rather than replacing it. The complete pipeline is evaluated both in isolation (per-model metrics on a shared test set) and end-to-end, where agreement between YOLOv8’s and ProtoTree’s classification on detector-predicted crops is used to assess how faithfully the prototype tree’s explanation reflects the detector’s own decision. Results show that detector accuracy keeps improving with larger YOLOv8 variants, but with rapidly diminishing marginal gains beyond the medium variant relative to their additional latency and memory cost. The prototype tree reaches 97.5% accuracy on ground-truth test crops, and—when applied to the detector’s own predicted regions—agrees with YOLOv8’s classification in over 93% of cases, indicating that it remains a faithful, human-readable, explanatory layer even on the detector’s imperfect, predicted bounding boxes.

Index Terms—Traffic sign detection, interpretable machine learning, explainable AI, YOLO, prototype-based classification, neural prototype tree, object detection, image classification.

I. INTRODUCTION

Traffic sign recognition is a core component of Advanced Driver Assistance Systems (ADAS) and autonomous driving pipelines, where timely and accurate detection of road signs directly impacts driver safety and regulatory compliance. A missed or misclassified speed limit, stop, or yield sign can lead to incorrect driving decisions, making the reliability of the underlying perception system a safety-critical requirement rather than a purely academic concern. Over the past decade, deep learning-based object detectors have become the dominant

approach to this task, with the YOLO family of models [1] establishing itself as a standard choice due to its favorable trade-off between detection accuracy and inference speed, both of which are essential for real-time, in-vehicle deployment.

At the same time, the increasing reliance on deep neural networks for safety-critical decisions has raised concerns about their lack of transparency. Standard convolutional classifiers, including those used as the classification head of object detectors, operate as black boxes [2]: they produce a class label and a confidence score, but offer no human-interpretable justification for the decision. This opacity is particularly problematic in the automotive domain, where regulatory bodies and end users increasingly demand explainable decision-making from AI-based systems, especially in cases of misclassification or system failure. The field of explainable AI (XAI) [3] has responded with two broad families of approaches: post-hoc explanation methods, which attempt to justify the decisions of an already-trained black-box model, and inherently interpretable models, which are designed from the outset to expose their reasoning process. Among the latter, prototype-based architectures such as ProtoTree [4] have shown that it is possible to retain competitive classification accuracy while structuring the decision process as a binary tree of learned visual prototypes, each of which can be visualized and inspected directly.

This work addresses both concerns jointly. Rather than treating detection accuracy and interpretability as competing objectives, we propose a two-stage pipeline in which a fine-tuned YOLOv8 detector [1] is responsible for localizing and classifying traffic signs in unconstrained driving scenes, while a ProtoTree classifier [4] is applied on the same detected regions to produce a transparent, case-based explanation of the sign category, alongside YOLOv8’s own prediction rather than in place of it. This separation of concerns allows each component to be optimized and evaluated according to criteria appropriate to its role: detection performance is measured through standard object detection metrics (mAP, precision, recall) under a rigorous model selection protocol, while the prototype tree is evaluated both for standalone classification

accuracy and for how closely its predictions track those of the detector it is meant to explain.

Several practical challenges arise when implementing such a pipeline on real-world data. Publicly available traffic sign datasets are often affected by severe class imbalance, incomplete or inconsistent annotations, or a mismatch between the data available for detection and classification tasks—an issue we encountered directly during dataset selection and address by adopting a large-scale, balanced detection dataset and a curated subset of classes for the interpretable classification stage. Model and hyperparameter selection for deep architectures is computationally expensive when performed naively; we adopt a staged selection strategy—single-fold screening for architecture and hyperparameter choices, followed by full cross-validation only on the final configuration—that balances methodological rigor with the computational constraints of the project.

By design, the prototype tree is trained exclusively on ground-truth crops rather than on the detector’s own predictions, decoupling the two models’ training processes and avoiding the propagation of localization errors into the classifier. The complete pipeline is nonetheless evaluated end-to-end, to quantify how often the two models agree when ProtoTree operates on the detector’s predicted, rather than ground-truth, regions.

The remainder of this paper is organized as follows. Section II reviews related work on traffic sign detection and interpretable image classification. Section III describes the dataset. Section IV details the experimental methodology, including model selection and training procedures for both YOLOv8 and ProtoTree. Section V presents detection, classification, and end-to-end pipeline results. Section VI discusses limitations and future work.

II. RELATED WORKS

A. Traffic Sign Detection

Recent deep learning frameworks for traffic sign detection predominantly focus on balancing raw accuracy and computational efficiency. Advanced YOLOv8 variants have introduced attention modules and multi-scale feature fusion to capture small, degraded targets under adverse conditions [5], or applied lightweight re-parameterization techniques to minimize memory footprints for edge deployment [6]. Other approaches isolate detection from classification by pairing YOLOv8 with decoupled deep backbones [7]. However, these methodologies share a critical vulnerability: they treat the classification pipeline as a complete black box. While achieving high benchmark metrics, their architectural refinements offer no safety guarantees or transparency regarding *why* a specific visual target was classified as a regulatory sign—a major barrier for safety-critical autonomous driving systems.

B. Interpretable Image Classification

In the field of explainable AI, methodologies generally fall into two main paradigms: post-hoc explanation methods,

which approximate the reasoning of an already-trained black-box model, and models that are interpretable-by-design. For safety-critical applications like autonomous driving, our work focuses on the latter paradigm to ensure that transparency is directly embedded into the decision-making architecture rather than computed after the fact as an approximation.

Within this domain, ProtoPNet [8] pioneered case-based reasoning by matching image regions to learned visual prototypes (“this looks like that”). However, traditional prototype networks often introduce significant feature redundancy and scale linearly during inference, limiting real-time applicability. ProtoTree [4] solves these issues by organizing prototypes into a strict binary decision tree, optimizing evaluation times via conditional routing while providing a traceable root-to-leaf path. Rather than relying on post-hoc tools, our work pairs YOLOv8 with ProtoTree as an inherently interpretable classification stage, directly securing the structural transparency of the detector’s visual decisions.

III. DATASET DESCRIPTION

The dataset used for training, validating, and testing the object detection model discussed in Section IV is based on the DFG Traffic Sign Dataset [9], a public dataset containing traffic signs from the Slovenian road network. The version employed in this work was obtained from the official distribution already enriched through data augmentation. Of the 16,264 images in the collection, 14,029 contain at least one annotated sign, for a total of 44,091 sign instances distributed across 200 distinct categories. These categories are taxonomically grouped into seven macro-classes: warning, prohibitory, mandatory, additional, special, tourist, and variable-message signs. The dataset is approximately balanced across the 200 classes, mitigating long-tailed distribution issues that are common in traffic sign benchmarks, as shown in Fig. 1.

Originally provided in COCO JSON format, the annotations (bounding box coordinates) were converted into the normalized YOLO format. Due to the multi-label nature of the images (which frequently contain multiple traffic signs), a multi-label stratification strategy was applied to split the dataset:

- **Test Set:** A fixed 15% partition (2,085 images, 6,635 sign instances) was isolated and reserved exclusively for the final model evaluation, never used during training, validation, or model selection.
- **Training and Validation Sets:** The remaining 85% of the annotated images was partitioned using a 5-fold stratified cross-validation scheme. In each run, one fold is used for validation, while the other four folds are used for training.

This partitioning strategy guarantees that all 200 classes are proportionally represented in every split, preventing training bias and ensuring a robust evaluation of model generalization. The same test split (at the image level) is shared by the YOLOv8 detector and, through ground-truth crop extraction, by the ProtoTree classifier, ensuring that the end-to-end pipeline evaluation in Section V is free of data leakage between the two models.

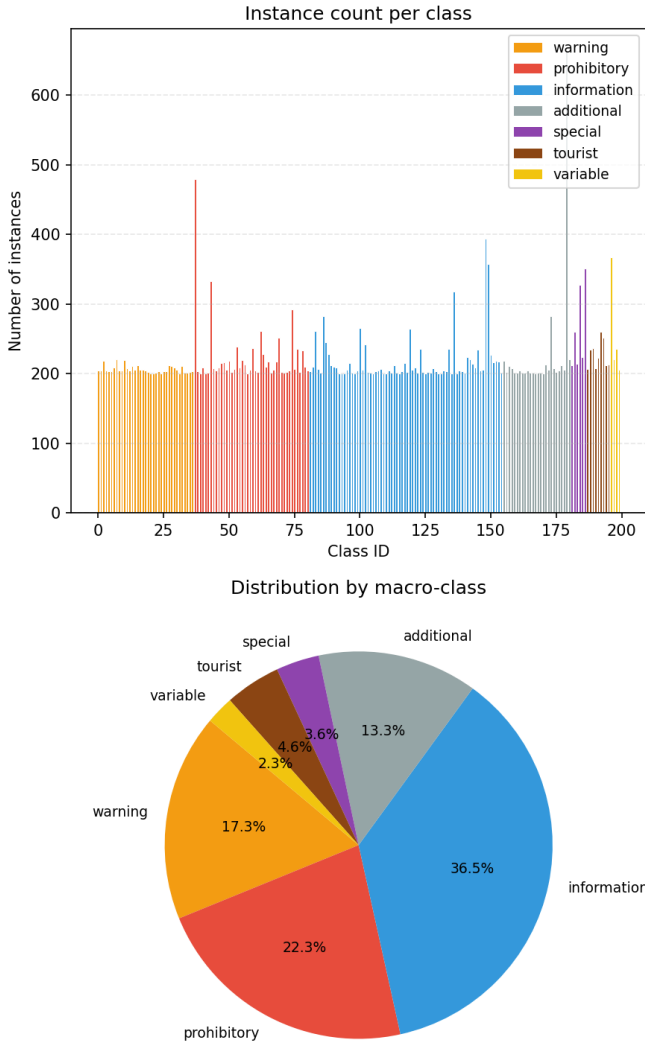


Fig. 1. Distribution of sign instances across the seven macro-classes and per-class instance counts.

IV. EXPERIMENTAL METHODOLOGY

This section details the experimental methodology adopted to evaluate the proposed two-stage architecture for traffic sign detection and interpretable explanation. We describe the model selection and training procedures for both the YOLOv8 detector and the ProtoTree classifier, along with the evaluation metrics used to assess each component.

A. Data Partitioning

Model selection and training rely on the dataset splits described in Section III, with a fixed held-out test set kept strictly unseen during training and a stratified 5-fold cross-validation scheme applied to the remaining data.

B. Object Detection and Primary Classification (YOLOv8)

The first stage of the pipeline relies on YOLOv8 to jointly perform object detection and primary classification. Given the

computational cost of training multiple deep architectures under full cross-validation, model and hyperparameter selection were performed in two single-fold screening phases, with full cross-validation reserved for the final configuration only.

In the first phase, five YOLOv8 variants—ranging from Nano to XLarge—were trained on a single fold (Fold 0) under identical default settings (initial learning rate 4.9×10^{-5} , batch size 16, maximum 50 epochs with early stopping, patience of 15 epochs) to evaluate the trade-off between detection accuracy, inference latency, and memory footprint. Candidate models were ranked according to mAP@50, inference latency, and model size, and the variant offering the most favorable accuracy-efficiency trade-off—YOLOv8m—was retained for subsequent tuning.

In the second phase, a localized grid search was conducted on the selected variant to optimize training hyperparameters, again on Fold 0. The search space covered two initial learning rates (the default 4.9×10^{-5} and a higher 1×10^{-3}) and two backbone freezing strategies (no frozen layers versus freezing the first 10 layers). The configuration with $lr_0 = 1 \times 10^{-3}$ and no frozen layers achieved the highest mAP@50 (0.929) and was selected as the final training setup.

Finally, in the third phase, the selected configuration was evaluated under a full 5-fold cross-validation scheme. To ensure that the training process reached true convergence—as the early stopping mechanism was never triggered during initial 50-epoch trials—the training limit was extended to 80 epochs. This cross-validation process ensures that the hyperparameter tuning and model selection are robust and independent of any specific validation split.

For the subsequent end-to-end pipeline evaluation, the weights corresponding to the fold with the highest validation mAP@50 were selected. This chosen model instance was then evaluated on the completely independent, unseen test set (as detailed in Section V), ensuring an unbiased and statistically reliable estimation of the system’s real-world performance.

C. Interpretable Classification (ProtoTree)

To provide a transparent, case-based explanation alongside the detector’s classification, a ProtoTree model was integrated into the pipeline. We extended the official repository [10] with a dedicated data-loading function to support our traffic-sign crops (the original codebase natively handles only the CUB and Stanford Cars datasets).

ProtoTree was trained exclusively on ground-truth crops extracted from 25 selected traffic sign classes, chosen both for their relevance to road safety and to keep the resulting decision tree compact and visually inspectable as a whole, rather than only through individual root-to-leaf paths. The training and test splits were aligned at the image level with those used for YOLOv8. Since the official ProtoTree training procedure does not employ a separate validation split, the training and validation images of the reference fold were merged into a single training set for crop extraction; the particular fold used for this purpose is immaterial, as training and validation images are pooled together regardless of which fold they

originate from, while the held-out test images remain shared and unseen, as described in Section III. Data augmentation was applied to this training subset only to improve feature robustness.

The architecture combines a ResNet-50 backbone, pre-trained on ImageNet, with a prototype layer that learns 256-dimensional prototypical features. Following the recommendations of the official ProtoTree implementation repository [10] to ensure stable convergence, the decision tree was configured with a maximum depth of 6, supporting up to 64 leaves—an optimized capacity that directly stems from our constrained 25-class selection. Training was carried out for 100 epochs, applying differentiated learning rates to balance feature adaptation and the preservation of pretrained representations: 1×10^{-5} for the backbone and 1×10^{-3} for the prototype layer, with the backbone frozen for the first 30 epochs. After training, the tree was pruned to remove rarely used branches, and the learned latent prototypes were projected onto their nearest real training patches, ensuring that every decision is grounded in an actual, inspectable visual example.

Classification performance was measured via overall accuracy, precision, and recall on the held-out test set, with confusion matrices used to identify class-specific weaknesses. These quantitative results were complemented by a qualitative assessment of the local explanations—i.e., the decision paths through the tree—produced for individual predictions.

D. End-to-End Pipeline Evaluation

Since ProtoTree is intended to explain, rather than replace, YOLOv8’s classification, the complete pipeline was evaluated by measuring *agreement* between the two models’ predictions. For every detection produced by YOLOv8 on the test set whose predicted class falls within the 25 classes covered by ProtoTree, the corresponding image region is cropped according to the detector’s predicted bounding box (not the ground-truth box) and classified by ProtoTree. The resulting class is compared directly against YOLOv8’s own predicted class for that same detection. A high agreement rate indicates that the prototype tree’s explanation is representative of the reasoning that plausibly underlies the detector’s decision, even though the two models are trained independently and on different inputs (full images for YOLOv8, ground-truth crops for ProtoTree). Agreement was computed both overall and per class, to identify categories for which the two models tend to diverge.

V. RESULTS

A. Detector Selection and Tuning

Table I summarizes the Phase 1 screening across the five YOLOv8 variants on Fold 0, and Fig. 2 visualizes the resulting accuracy-speed trade-off. Accuracy improves monotonically with model size, but with clearly diminishing returns from the selected variant onward: moving from Medium to Large to XLarge yields a combined gain of only +0.026 mAP@50 (0.908 to 0.934) at the cost of a 24% increase in inference latency (16.49 ms to 20.46 ms) and more than double the

model size, whereas the earlier gain from Nano to Medium was substantially larger (+0.197 mAP@50) for a negligible latency increase. YOLOv8m was therefore selected as offering the best accuracy-efficiency trade-off, since further increases in model size beyond this point are not justified by the corresponding accuracy gain.

TABLE I
PHASE 1: YOLOV8 VARIANT COMPARISON ON FOLD 0

Variant	mAP@50	mAP@50-95	Prec.	Rec.	Lat. (ms)
n	0.711	0.651	0.701	0.677	14.94
s	0.846	0.787	0.832	0.793	14.99
m	0.908	0.858	0.896	0.858	16.49
l	0.930	0.887	0.923	0.884	18.08
x	0.934	0.893	0.928	0.892	20.46

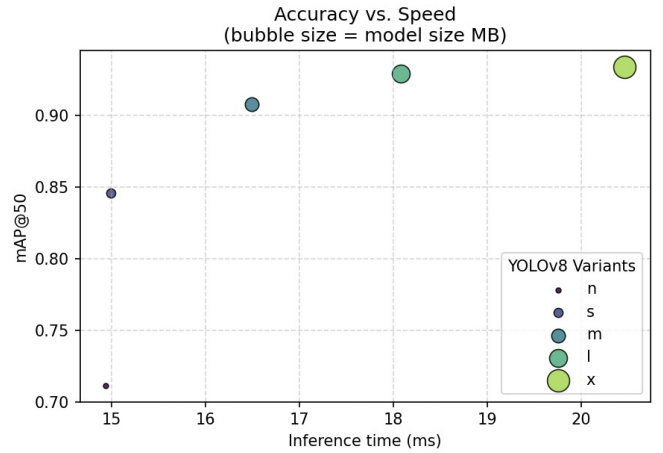


Fig. 2. Accuracy-speed trade-off across YOLOv8 variants on Fold 0. The curve visibly flattens from the Medium variant onward, illustrating the diminishing returns discussed above.

Table II reports the Phase 2 grid search on YOLOv8m. The configuration with the higher learning rate (1×10^{-3}) and no frozen layers consistently outperformed the more conservative alternatives, suggesting that, despite the relatively small per-class sample size, full fine-tuning of the pretrained backbone remains beneficial for this dataset.

TABLE II
PHASE 2: HYPERPARAMETER GRID SEARCH ON YOLOV8M, FOLD 0

lr_0	Freeze	mAP@50	mAP@50-95	Recall
4.9×10^{-5}	—	0.908	0.858	0.858
4.9×10^{-5}	10 layers	0.841	0.789	0.786
1×10^{-3}	—	0.929	0.884	0.883
1×10^{-3}	10 layers	0.872	0.818	0.815

Table III reports the Phase 3 cross-validation results for the selected configuration across five folds. The low standard deviation (mAP@50 std. of 0.0028) indicates that the configuration’s performance is stable across different training subsets, supporting the reliability of the selection.

TABLE III
PHASE 3: 5-FOLD CROSS-VALIDATION OF THE FINAL YOLOv8m
CONFIGURATION OVER 80 EPOCHS

Metric	Mean $\pm$ Std.
mAP@50	0.9318 $\pm$ 0.0028
mAP@50-95	0.8877 $\pm$ 0.0035
Precision	0.9294 $\pm$ 0.0051
Recall	0.8910 $\pm$ 0.0054

Evaluated once on the held-out test set, the model from the best-performing fold achieved mAP@50 = 0.932, mAP@50-95 = 0.892, precision = 0.933, and recall = 0.887, consistent with the cross-validated estimate above and confirming the absence of significant overfitting to the validation folds used for model selection. Fig. 3 reports the corresponding precision-recall curve on the test set, summarizing the detector’s behavior across confidence thresholds in a single view.

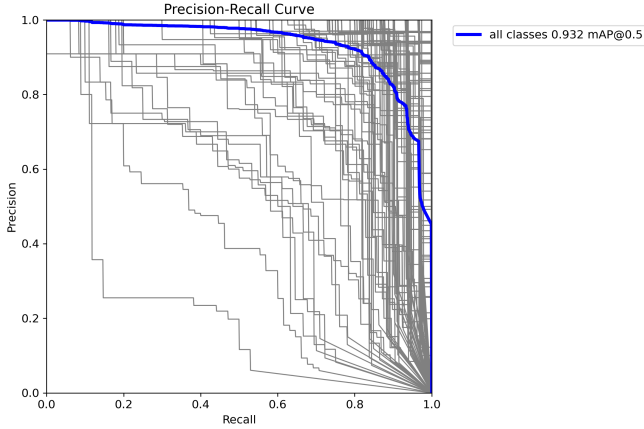


Fig. 3. Precision-recall curve of the final YOLOv8m model on the held-out test set, across confidence thresholds.

To further characterize the detector’s behavior on the classes relevant to the explanatory stage, Fig. 4 shows the confusion matrix restricted to the 25 classes shared with ProtoTree.

B. ProtoTree Classification

On the 25 selected classes, ProtoTree reached a peak test accuracy of 97.5% during training (97.4% at the final epoch), evaluated on ground-truth crops. Fig. 5 shows the resulting pruned and projected tree in its entirety, illustrating how each prediction follows a traceable, visually grounded root-to-leaf path. The per-class classification report on the test set (874 crops) confirms high and largely uniform performance, with an overall accuracy of 0.97 (macro-averaged precision and recall both 0.97). The lowest-performing class reached a recall of 0.73, indicating that a small number of categories remain harder to disambiguate even with clean, ground-truth input crops; the confusion matrix in Fig. 7 identifies the specific class pairs involved. Fig. 6 illustrates this traceable reasoning process for a single test prediction.

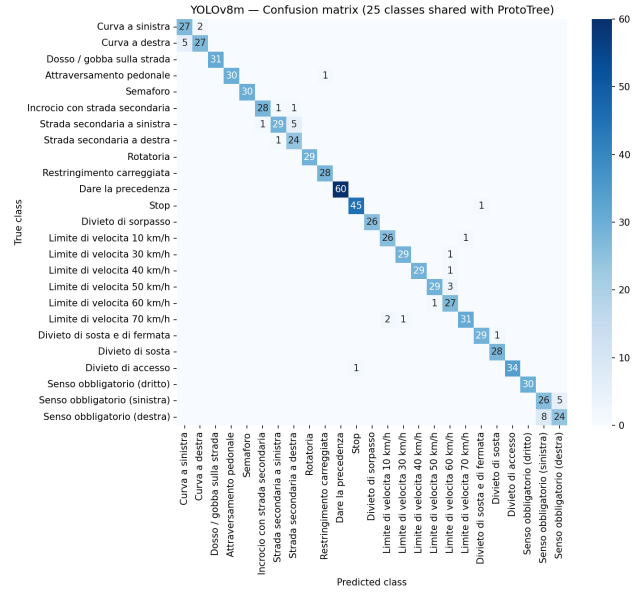


Fig. 4. YOLOv8m confusion matrix on the test set, restricted to the 25 classes shared with ProtoTree, for direct comparison with Fig. 7.



Fig. 5. Pruned and projected ProtoTree: each internal node shows the projected visual prototype; routing follows the presence/absence of that prototype in the input crop.

C. End-to-End Agreement

On the test set, restricting to YOLOv8 detections whose predicted class falls within ProtoTree’s 25-class scope (842 detections), ProtoTree’s classification on the detector-predicted crop agreed with YOLOv8’s own classification in 93.2% of cases (785/842). Per-class agreement varied considerably, from 100% for several well-separated sign categories down to 66.7% for the weakest class, mirroring, at least in part, the classes that were already the hardest for ProtoTree in isolation. This indicates that the prototype tree, despite being trained independently and only on clean ground-truth crops, remains a largely faithful explanatory proxy for YOLOv8’s classification even when operating on the detector’s imperfect, predicted bounding boxes, while also surfacing specific categories where the two models’ decision boundaries diverge and a practitioner should treat the explanation with more caution. Figs. 8–9 illustrate this end-to-end process on a single representative example, from raw detection to the corresponding prototype-based explanation.

VI. DISCUSSION AND FUTURE WORK

The results support the central premise of this work: detection and interpretability need not be pursued as competing objectives within the same model. By delegating localization

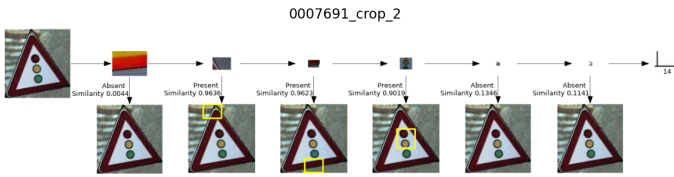


Fig. 6. Example local explanation: Traffic light.

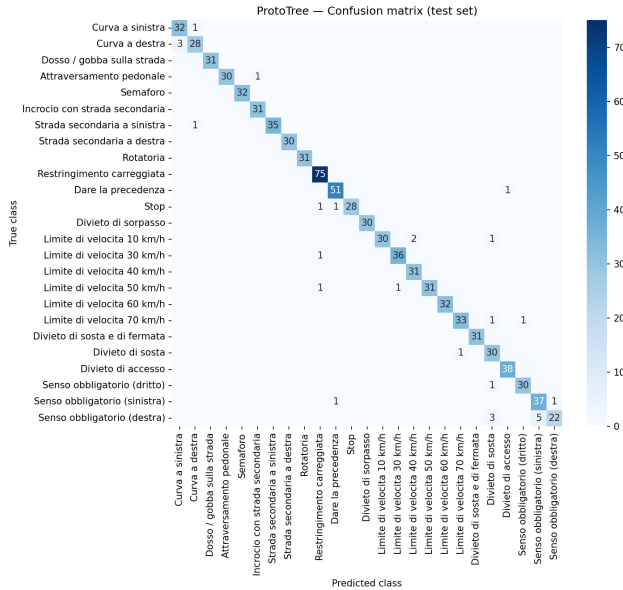


Fig. 7. ProtoTree confusion matrix on ground-truth test crops (25 classes).

and primary classification to a tuned YOLOv8 detector and attaching a compact, independently trained ProtoTree as an explanatory layer, we obtain a system whose detection performance is unaffected by the interpretability requirement, while still providing visually-grounded, traceable justifications for a relevant subset of its classifications.

Several limitations should be acknowledged. First, ProtoTree was trained on a 25-class subset chosen for road-safety relevance and explanatory tractability; extending interpretable coverage to all target classes would require either a substantially deeper tree—at the cost of visual inspectability—or an ensemble of smaller, topic-specific trees. Second, in the end-to-end evaluation, the agreement between the two models is inherently constrained by the detector’s performance. When YOLOv8 produces a false positive or an inaccurate bounding box (e.g., localizing background noise or non-sign objects), the downstream ProtoTree classifier is forced to categorize a corrupted crop, leading to a structural drop in pipeline consistency that stems from localization errors rather than classification flaws. Third, to optimize computational resources, model selection during Phases 1 and 2 was deliberately conducted on a single data split, reserving the full 5-fold cross-validation exclusively for the final verification in Phase 3. Although Phase 3 successfully confirmed the high stability and low



Fig. 8. End-to-end pipeline example (Stage 1): Object detection and traffic sign localization performed by the YOLOv8 model on the input image.

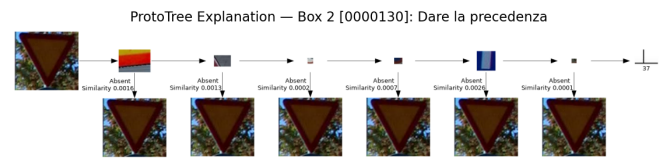


Fig. 9. End-to-end pipeline example (Stage 2): ProtoTree local explanation path generated for the traffic sign localized in Fig. 8, illustrating the sequence of prototype decisions.

variance of the chosen model, adopting a single-fold proxy in the early stages remains a procedural trade-off; running a full cross-validation from the very beginning would have provided an even more consistent and unbiased framework for the initial architecture selection. Fourth, due to practical time constraints, our hyperparameter optimization was focused on macro-architectural variants rather than an exhaustive grid search. Consequently, wider combinations of fine-grained parameters—such as different batch sizes, alternative learning rates, or specific structural settings for ProtoTree—were not systematically explored.

Future work will address these limitations along four main directions. First, we aim to extend the interpretable classification stage to a larger fraction of traffic sign classes without sacrificing visual tractability, exploring a hierarchical arrangement of smaller prototype trees organized by macro-class. Second, we plan to incorporate advanced filtering mechanisms between the two stages to prevent ProtoTree from processing low-confidence or poorly localized detector crops, thereby shielding the classifier from upstream localization noise. Third, future architectural evaluations will employ a full cross-validation scheme from the very first phase to ensure a perfectly consistent and unbiased selection framework throughout the entire optimization process. Finally, we intend to conduct an exhaustive grid search to systematically explore finer hyperparameter combinations, including varying batch sizes, alternative learning rates, and specific structural configurations for the prototype layers.

REFERENCES

- [1] G. Jocher, A. Chaurasia, and J. Qiu, “Ultralytics YOLO,” <https://github.com/ultralytics/ultralytics>, 2023.
- [2] R. Guidotti, A. Monreale, S. Ruggieri, F. Turini, F. Giannotti, and D. Pedreschi, “A survey of methods for explaining black box models,” *ACM Computing Surveys*, vol. 51, no. 5, pp. 1–42, 2019.
- [3] A. Kuznetsov, B. Gyevnar, C. Wang, S. Peters, and S. V. Albrecht, “Explainable AI for safe and trustworthy autonomous driving: A systematic review,” *IEEE Transactions on Intelligent Transportation Systems*, vol. 25, no. 12, pp. 19 342–19 364, 2024.
- [4] M. Nauta, R. van Bree, and C. Seifert, “Neural prototype trees for interpretable fine-grained image recognition,” in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2021, pp. 14 933–14 943.
- [5] B. Ji, J. Xu, Y. Liu, P. Fan, and M. Wang, “Improved YOLOv8 for small traffic sign detection under complex environmental conditions,” *Franklin Open*, vol. 8, p. 100167, 2024.
- [6] M. Hussein and W.-X. Zhu, “A real-time ghost machine learning model built on YOLOv8 for traffic road signs detection and classification in germany,” *Multimedia Systems*, vol. 30, p. 344, 2024.
- [7] Aryan, A. J. Mahariba, B. Joshi, D. Singh, and R. S. Chhabra, “Enhanced traffic sign detection and classification using YOLOv8x and SEResNet-101x model: A deep learning approach,” in *Computer Vision and Image Processing (CVIP 2024)*, ser. Communications in Computer and Information Science, vol. 2474. Cham: Springer, 2026.
- [8] C. Chen, O. Li, D. Tao, A. Barnett, C. Rudin, and J. K. Su, “This looks like that: Deep learning for interpretable image recognition,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 32, 2019, pp. 8930–8941.
- [9] D. Tabernik and D. Skočaj, “Deep learning for large-scale traffic-sign detection and recognition,” *IEEE Transactions on Intelligent Transportation Systems*, vol. 21, no. 4, pp. 1427–1440, 2020, dataset available at <https://www.vicos.si/resources/dfg/>.
- [10] M. Nauta, “ProtoTree: Neural prototype trees for interpretable fine-grained image recognition,” <https://github.com/M-Nauta/ProtoTree>, 2021.